Night Flash Edge

A Compact Unpaired Style-Transfer System for a Disposable-Camera Aesthetic

Technical Report

September 27, 2026

Abstract

This report documents a compact learned image-translation model intended to reproduce a disposable-camera / Night Flash aesthetic inside an existing native camera application. The system is deliberately small enough to run in real time on a phone and to export to Core ML, and it learns from *unpaired* image domains, so no aligned before/after photographs are required.

The report covers the architecture and its three switchable capabilities; the unpaired training objective and a correction to it that materially changed training behaviour; a synthetic degradation pipeline that manufactures a source domain from the target photographs themselves, in two variants with different guarantees; a held-out validation regime with reproducible strips, a metric history and best-checkpoint selection; the inference path including optional tiled and downscaled modes; and the measurement results behind each of these decisions. Measurements quoted throughout were produced by the code in this repository and are reproducible with the scripts referenced in §9.

Contents

1 Problem statement and constraints **3**
- 1.1 What the model must do . . . 3
- 1.2 Why unpaired . . . 3
- 1.3 Deployment boundary . . . 3

2 Architecture **3**
- 2.1 Generator . . . 3
- 2.2 Bounded residual, and why it is the load-bearing decision . . . 4
- 2.3 Output heads . . . 4
- 2.4 Three capabilities . . . 4

3 The training objective **5**
- 3.1 Unpaired baseline . . . 5
- 3.2 A correction: where the texture penalty belonged . . . 5
- 3.3 Supervised reconstruction, where a correspondence exists . . . 6

4 Synthetic degradation **6**
- 4.1 The problem it solves . . . 6
- 4.2 Pipeline and ordering . . . 6
- 4.3 Measured effect . . . 6
- 4.4 Two honest limitations . . . 6
- 4.5 Variation . . . 7

5 Training modes **7**
- 5.1 Three ways to form a pair . . . 7
- 5.2 `sameset` removes a shortcut . . . 7
- 5.3 Mode resolution . . . 8
- 5.4 A shared-warp bug worth recording . . . 8

6 Held-out validation **8**
6.1 Why a strip and not a number . . . 8
6.2 Layout . . . 8
6.3 Two determinisms, and why both are required . . . 8
6.4 Splitting . . . 9
6.5 Metric history and checkpoint selection . . . 9
6.6 Exponential moving average . . . 9

7 Inference **9**
7.1 Whole frame, the default . . . 9
7.2 Tiled inference . . . 9
7.3 Downscaling, and the training/inference gap . . . 10

8 Core ML boundary **10**

9 Verification **10**
9.1 Test suite . . . 10
9.2 Degradation probes . . . 11
9.3 Practical guidance . . . 11

1 Problem statement and constraints

1.1 What the model must do

Given an ordinary photograph, produce one that reads as a disposable-camera or Night Flash capture: hard direct flash on a near subject, rapid luminance falloff to a dark surround, crushed shadows, warm ambient tint, visible film grain and light leak. Two properties are non-negotiable:

1. **Geometry, faces and text must survive.** The model is a filter, not a generator. A face that changes shape is a defect, not a style.
2. **It must run on a phone, in real time.** That caps the parameter budget at single-digit millions and forbids any heavyweight auxiliary network in the shipped graph.

1.2 Why unpaired

Genuine before/after pairs of the same scene do not exist. The style lives in lighting and film response, not in content, so the two domains share *content* statistics only loosely. This is unpaired domain translation in the CycleGAN sense, and it imposes a structural constraint on the loss: nothing in the objective may compare a generated image to a specific target pixel.

1.3 Deployment boundary

The application already owns the Vision person mask and the Core Image / Metal finishing chain. The learned model is inserted before that chain and is responsible for the learned appearance only. Grain, leak and vignette are rendered by a seeded procedural stage that can be disabled at export time, which keeps the deterministic and stochastic paths independently testable.

2 Architecture

2.1 Generator

The default architecture, `edgehat`, is a convolutional encoder/decoder with two kinds of attention.

`EdgeViT` blocks
: Multi-head self-attention confined to local windows. Linear in image size, and the window can be precomputed at construction time so nothing depends on a traced tensor shape.

HAT block
: One cross-attention stage with overlapping shifted windows, giving a global receptive field for the price of a single block.

Level widths are $48 \to 96 \to 192 \to 384 \to 384$, with a single fusion at the third stage. The total is **4.02 M parameters** for the `freqsplit` head.

Architecture	Attention	Params	Role
`edgehat`	local windows + one HAT block	4.02 M	default
`swinir`	windowed, relative position bias	4.92 M	comparison arm
`restormer`	over channels	0.76 M	legacy
`hybrid`	pooled tokens	0.76 M	legacy, keeps old checkpoints

Table 1: Architectures. The two legacy entries do not honour the newer heads and exist only so pre-existing checkpoints remain loadable.

2.2 Bounded residual, and why it is the load-bearing decision

The network predicts a residual, and the output is

$$x_{\text{out}} = \text{clamp}(x_{\text{in}} + \Delta(x),\, 0,\, 1)\,, \tag{1}$$

with $\|\Delta\|_\infty$ capped by `residual_limit`.

This single constraint is what protects identity. The model is structurally incapable of inventing content, because the only way to produce a pixel is to perturb the pixel that was already there. It also means an under-trained model degrades to a near-identity filter rather than to noise, which is the correct failure direction for a camera filter.

2.3 Output heads

The head decides what the network is permitted to do.

Head	Learns	Params	Character
`tanh`	bounded delta	21 k	safest with little data
`perpixel`	per-pixel delta, clamped on read	42 k	free to repaint pixels
`freqsplit`	low / mid / high bands separately	63 k	grain and halation in the high band
`lut3d`	33^3 trilinear LUT + per-pixel	129 k	global grade and local detail
`lut3d+freqsplit`	LUT and band split	192 k	recommended
`lut3d+perpixel`	LUT and per-pixel	171 k	
`tanh+freqsplit`	bounded and band split	84 k	everything bounded

Table 2: Heads and composites. `perpixel+freqsplit` is deliberately absent: both are per-pixel, so it costs parameters without adding a distinct inductive bias.

The default is `freqsplit` rather than a pure per-pixel head because grain and halation live in the high frequency band, which a per-pixel parameterisation represents poorly. Splitting the residual by frequency lets each band be predicted at the resolution it needs.

2.4 Three capabilities

Each is independently switchable and each is gated by a *zero-initialised* mix scalar, so a freshly built model is exactly the plain residual and the extras can only engage once training gives them a reason to. That is what makes them safe to enable by default.

Halation

A physical operator, not a learned guess:

$$\begin{aligned} \ell &= Y \cdot (0.2126,\, 0.7152,\, 0.0722) \\ h &= \text{relu}(\ell - \tau) \,/\, (1 - \tau) \\ s &= \text{DilateBlur}(h) \\ g &= \text{relu}(s + T(F)) \odot \mathbf{tint} \end{aligned} \tag{2}$$

Light scattering through the film base is a low-pass operation on over-threshold luminance, so a dilated depthwise blur is both faithful and very cheap. A learned convolution stack would be strictly worse: more parameters, less physics, and no ability to generalise to a highlight the training set never contained.

Subject gate

A soft $S(x, y) \in [0, 1]$ from mid-level features. When a Vision person mask is supplied it is adaptively pooled to the same grid and added, so the gate is *guided* rather than guessed. Tone is then modulated as $\Delta + \lambda S \tanh(\Delta)$, letting the subject diverge from the background without either running away.

Region exposure
Pools to a 4×4 grid and applies a 1×1 convolution, so each cell is conditioned on its own features. Flattening the grid into one linear layer would couple every cell to every other.

Interaction with tiled inference. Region exposure conditions each cell on that cell's own features, which assumes the grid spans the photograph. Under tiling it spans a tile. `Session.infer` prints a note when it detects tiling together with region exposure; either infer whole-frame or train with `--no-region-exposure`.

3 The training objective

3.1 Unpaired baseline

The generator is trained against a PatchGAN discriminator with a set of constraints:

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}} + 1.5\,\mathcal{L}_{\text{content}} + 0.35\,\mathcal{L}_{\text{moment}} + 0.75\,\mathcal{L}_{\text{identity}} + 0.02\,\mathcal{L}_{\text{effect}}. \tag{3}$$

The discriminator is the only term that sees targets as targets. Everything else is a leash. These weights are fixed and are not exposed on `nf` or `Session`; the legacy `nightflash-train` has flags for them.

3.2 A correction: where the texture penalty belonged

The content term is a convex combination of an L_1 half and a Sobel edge half. In the original implementation both halves were evaluated on the *post-effect* image x_{styled}, the output after grain and halation had been applied.

That is the wrong place for the L_1 half, and measurably so. L_1 is texture sensitive, so comparing a grainy image against a clean source charges the output for the grain. Measured on a trained model:

Quantity	Value
$\text{content}(x_{\text{det}}, x_{\text{src}})$	0.02271
$\text{content}\big(x_{\text{styled}}, x_{\text{src}}\big)$	0.04565
Penalty the stochastic effects incur	+0.02294
$\|x_{\text{det}} - x_{\text{src}}\|_1$	0.01233

The robust claim is the ratio, not the absolute values, which are run-specific. The effects were charged *1.86 times* more than the entire learned residual was charged for performing the whole style transfer. With the adversarial term at weight 1.0 asking for grain and the content term at weight 1.5 penalising it, the content term simply won, and the model learned to suppress the grain and halation the effect head exists to produce. This is the classical failure where an L_1 or L_2 penalty on a stochastic target averages the stochastic component out of both sides.

The fix. Split the two halves across different images:

$$\mathcal{L}_{\text{content}} = 0.65 \cdot L_1(x_{\text{det}},\, x_{\text{src}}) + 0.35 \cdot \mathcal{E}\big(x_{\text{styled}},\, x_{\text{src}}\big)\,. \tag{4}$$

The L_1 half now compares the *pre-effect* image, so stochastic grain is not charged. The edge half still compares the *styled* image, because structure genuinely should be verified on the final output, and a Sobel response is indifferent to fine texture while being sensitive to broken geometry.

Consequence. Grain and halation become visibly stronger, which is the intent. It also means checkpoints trained under the old balance are not comparable to new ones, so `LOSS_VERSION` was incremented to 2 and is written into `config.json` and every checkpoint. Resuming a checkpoint whose recorded version differs emits a warning rather than silently mixing two objectives.

3.3 Supervised reconstruction, where a correspondence exists

When the training mode supplies a known corresponding target, one more term is available:

$$\mathcal{L}_{\text{paired}} = L_1(x_{\text{det}},\, y) + 0.5 \cdot \mathcal{E}(x_{\text{det}},\, y)\,, \tag{5}$$

evaluated on the *deterministic* output. Pointing an L_1 term at a stochastic target would re-create exactly the problem Eq. (4) exists to avoid, so it is not done.

4 Synthetic degradation

4.1 The problem it solves

Unpaired translation needs a source domain. When only a target domain is available, the source has to be manufactured. This module simulates what the same scene would have looked like through a non-flash camera.

4.2 Pipeline and ordering

The stages run in the order a camera pipeline would be *undone*. Resampling happens first so that photometric operations act on clean pixels.

#	Stage
1	crop and small rotation (shared with the target)
2	**downscale then upscale** — removes grain and fine detail
3	Gaussian blur — residual softening
4	contrast compression about mid-grey
5	black lift, $x \mapsto x + b(1-x)$, so white stays white
6	exposure gain
7	white-balance shift, warmed
8	saturation increase
9	light sensor noise

Stages 4 and 5 carry most of the domain shift: compressing the tonal range is what flattens the flash falloff signature, and it is the stage that makes the source look like an evenly-lit non-flash capture rather than a dark one.

4.3 Measured effect

On a real photograph, averaged over eight draws per profile:

	`subtle`	`moderate`	`strong`
high-frequency energy (relative)	0.52	0.30	0.21
tonal range $p_{99} - p_1$	0.76	0.62	0.58
saturation (absolute)	0.04	0.08	0.12
warmth $R - B$ (absolute)	0.040	0.078	0.121

4.4 Two honest limitations

Resampling cannot remove a wide halo. An early draft of this document claimed that resampling destroys halation bloom. Measurement refutes it. Halation is a wide, *low-frequency*

phenomenon, and a low-pass filter has no problem to solve: blurring a glow makes it brighter by spreading the core into the ring. On a synthetic probe with a specular point and a halo, the ratio of ring luminance to peak luminance *rises* from 0.57 to 0.86 as blur goes from 0 to 4. What the pipeline achieves is attenuation, driven by the tonal compression rather than the filtering, and that is sufficient for the purpose: the model sees a highlight with less glow than the target has and learns the difference. It would not be sufficient if the source needed to contain *zero* halation, which would require re-rendering the lighting rather than filtering it.

The source must remain plausible. A first `strong` preset used blur up to 2.5 and black lift up to 0.20. The result looked foggy rather than like a photograph, and its added noise was glaring against the softened background. Training on that teaches a model to expect an input distribution that never occurs at inference, which is worse than a weaker domain shift. The ranges were pulled back until `strong` still reads as a believable overcast capture.

4.5 Variation

Three independent levels, plus two optional mechanisms:

1. **Per image** — every parameter drawn independently per item.
2. **Per epoch** — free. The dataset seeds its generator with the epoch index, so the draw differs every epoch with no extra machinery.
3. **Per run** — the run seed shifts the whole stream.
4. **Identity path**, probability `none_prob` (default 0.10): the target is fed through undegraded. This anchors the identity case and widens the input distribution, which is the main defence against the failure mode in §5.2.
5. **Curriculum** (on by default): the profile ramps `subtle` → `moderate` → `strong` across training. Early epochs get an easy mapping, later epochs get robustness.

5 Training modes

5.1 Three ways to form a pair

`train_mode`	Source	Target	Correspondence
`unpaired`	`source_dir`	`target_dir`	none, cross-domain
`paired`	$\deg(x_i)$	x_i	exact, same file
`sameset`	$\deg(x_j)$, $j \neq i$	x_i	none, same pool

In both degradation modes the *shared geometric warp is applied first, to both images*, and the degradation runs afterwards on the source only. That ordering is what keeps the two halves consistent: any crop, flip or rotation is shared, and only the simulated camera differs.

5.2 `sameset` removes a shortcut

In `paired` mode a shortcut is available to the network: learn to invert *this specific* corruption. The cheapest way to reduce the loss is then $\deg^{-1}$, and the model never learns anything about flash physics that generalises. It will look excellent on freshly-degraded images and poor on a real photograph, and the failure is invisible unless you look for it.

In `sameset` that shortcut does not exist. x_i and x_j are unrelated photographs, so there is no pointwise relationship to invert and the model must learn the conditional distribution of the look instead.

The tell. If the model produces good results on freshly degraded images but poor results on ordinary photographs, it has learned the inverse rather than the aesthetic. The validation strip of §6 makes this visible within a few epochs.

The cost. Without a correspondence, `sameset` cannot report a real quality number. Only distribution-level metrics apply, and `best.pt` is deliberately not written, because ranking checkpoints on a PSNR that compares two unrelated photographs would be meaningless.

5.3 Mode resolution

`train_mode` defaults to `None` and is resolved from the arguments actually supplied: a single pool resolves to `paired`, two distinct pools resolve to `unpaired` with a printed notice. An explicitly requested mode is always honoured and rejected loudly when it cannot work. This keeps `paired` as the default for new single-pool work without breaking existing two-pool scripts.

5.4 A shared-warp bug worth recording

The first implementation of `make_pair` sampled scale, rotation and flips once and then applied them to the source and the target in two separate calls. The crop *offset*, however, was drawn from the shared random stream inside each call, so the two images received different offsets. This is harmless in `unpaired` mode, where the images are unrelated anyway. In `paired` mode it destroys the entire point: the correspondence.

The offset is now sampled once as a *fraction* and clamped independently per image, so two differently sized images still receive the same relative crop. Verified: an identical input image now yields byte-identical source and target in 60 of 60 trials.

6 Held-out validation

6.1 Why a strip and not a number

A loss curve shows the objective is decreasing. It does not show whether the output still looks like a photograph. One strip image per run, rendered every epoch, is cheap and is the thing a person actually looks at.

6.2 Layout

One image per row, three cells per image, ordered `input` | `output` | `target`. Rows rather than columns so a given image's input and output are adjacent and the comparison needs no eye travel. The header carries the epoch, the mode, the active degradation profile and the aggregate metrics.

6.3 Two determinisms, and why both are required

***Which* images appear.**
`val_select="fixed"` (the default) keeps the same set every epoch so per-image progress is readable. `"random"` answers a different question — is the model still healthy on *arbitrary* images — and hides the failure where a model looks great on a handful it has memorised.

***How* each is degraded and seeded.**
Always pinned, unconditionally. Seeds derive from the filename via SHA-256, not from the epoch-dependent stream, so a strip rendered in one session matches one rendered last week.

The second is not optional. A validation image degraded with a fresh random draw each epoch is a different problem each time, and any apparent improvement is noise. Python's `hash()` is salted per interpreter and therefore unusable here; a cryptographic digest is required for cross-session stability.

6.4 Splitting

Validation images are drawn from the target pool by a deterministic, seeded split and are never trained on. An explicit `--val-holdout-dir` overrides the fraction. A pool too small to split leaves the validation half empty rather than silently stealing training data.

6.5 Metric history and checkpoint selection

One CSV row per epoch: `epoch, images, psnr, ssim, l1, effect`. This serves two purposes. It plots the curve, and more importantly it makes checkpoint selection possible at all: before this, only `latest.pt` existed, so a run that peaked at epoch 40 and decayed by epoch 60 had no way to recover the good weights.

- `best.pt` is written whenever validation PSNR improves, in `paired` mode only, for the reason in §5.2.
- `keep_last` prunes older per-epoch checkpoints. Sixty epochs of a 17 MB checkpoint is a gigabyte, which fills Kaggle's disk and makes every snapshot slow.

6.6 Exponential moving average

`ema_decay` maintains a shadow copy of the generator weights and is checkpointed and resumed with the run. Averaging over the trajectory is standard for adversarial training and reduces frame-to-frame flicker, which matters directly for a camera preview.

7 Inference

7.1 Whole frame, the default

The image is letterboxed to `image_size`, styled, and restored to its exact original dimensions. Nothing is cropped and nothing is stretched. Verified landmark round-trip: worst positional error 0.0017, output size identical to input, reconstruction error 0.02–0.03 out of 255.

7.2 Tiled inference

Optional, for native-resolution detail. Overlapping tiles are crossfaded with a raised-cosine window.

Match the tile to the training crop. This is not a preference. A model trained on 256 px crops has learned 256 px-crop feature statistics; a different tile size is a distribution shift. `Session.infer` warns on a mismatch rather than blocking, because the mismatch is occasionally useful.

Tile	Overlap ratio	Tiles	MP	Note
256	25%	336	22.0	matches a 256 px run
256	50%	713	46.7	default; smoother
512	25%	88	23.1	matches a 512 px run
1024	25%	20	21.0	matches a 1024 px run

Table 3: Tiled cost on a 4032×3024 source. What matters is the overlap *ratio*: the three 25% rows all land near 22 MP, and going to 50% roughly doubles it.

Borders do not fade. A Hann window tapers to zero at its edges, so a naive crossfade fades the frame border towards black. Two mechanisms prevent it, so no pre-padding is required: the window carries a positive floor of 10^{-3}, and edge tiles extend past the frame with the chosen

padding, so the frame border is never a tile border. Verified with a 255-valued border strip, which survives at 254.6/255 with a worst-case error of 1.0; a fade would show near zero.

The blend uses the same window weight in numerator and denominator. Clamping only the denominator scales the output, which turned all four corners black in an earlier version.

7.3 Downscaling, and the training/inference gap

`max_edge` shrinks the long edge while preserving aspect ratio, before tiling. It never upscales. Aspect ratio is preserved to within 0.005 on the sizes that matter.

Degradation modes narrow the *input* distribution to degraded night photographs, which makes learning easier but can widen the gap to a real user photograph. Three mitigations are in place, in order of importance: wide per-image randomisation, the identity path, and the `strong` profile. An explicit inference-time input normalisation was considered and *rejected*: it risks double-degrading a user image, and the training-side mitigations achieve the same without inference cost or risk.

8 Core ML boundary

Export produces a `.mlpackage` or `.mlmodel`. All 28 architecture × head combinations have been verified to convert, with tensor-I/O mean absolute error 0.00030 against PyTorch.

Several implementation choices exist specifically because of this boundary, and should not be "cleaned up":

- `nn.MultiheadAttention` is unused. It lowers to a fused operator that fails conversion; attention uses explicit query/key/value matmuls.
- `Lut3DHead.table` is a flat $(s^3, 3)$ tensor. Per-channel gathers break Core ML scatter lowering.
- `grid_sample` is unused anywhere. It returned near-centre values regardless of the grid in testing, and the 3D LUT uses explicit trilinear gathers.
- `HATBlock.self.grid` and the `EdgeAttention` window clamp use Python integers precomputed in `__init__`. Under `torch.jit.trace`, `x.shape[1]` becomes a zero-dimensional tensor, which breaks `max`, `min` and `adaptive_avg_pool2d`.

9 Verification

9.1 Test suite

264 tests pass. Beyond shape and export checks, the suite pins the behaviours that would otherwise degrade silently:

- the content-loss split, including that the L_1 half is insensitive to the styled image and the edge half is not;
- that the shared warp yields byte-identical source and target from an identical input;
- that validation degradation is invariant across epochs, and *is* variant across them for training;
- that `sameset` never pairs an image with itself, and that a single-image pool falls back to `paired` with a warning rather than reaching `randrange(0)`;
- that `best.pt` is not written in `sameset`;
- an AST check that no module unpacks a generator call without `run_generator`, which would break every newer architecture.

9.2 Degradation probes

Three scripts were used to derive the numbers in this report and are the basis for the claims in §4:

- a stage-isolation probe, confirming each stage moves luminance in the intended direction;
- a halo probe, which is what falsified the claim that resampling removes halation;
- a real-photograph probe, reporting high-frequency energy, local contrast, tonal range, saturation and warmth per profile.

9.3 Practical guidance

Target domain first.
A PatchGAN learns whatever distribution the target folder contains. If that folder is wrong, no architecture, schedule or tiling will help, and the failure is silent. Measure the target domain's mean luminance before spending GPU hours; for a Night Flash target it should sit below 0.20, and daylit photography sits above 0.35.

Watch the strip, not the loss.
The discriminator loss is not a quality signal.

Two arms, two GPUs.
At 4 M parameters a single T4 is not saturated and data parallelism would add synchronisation cost without throughput. Two independent runs answer a real question that one run cannot.

No TPU.
The pipeline uses explicit gathers, `torch.jit` tracing for export, and a PatchGAN. A TPU would require an XLA rewrite of the augmentation path, and Core ML export would still happen on CPU.